

CSE 220 – DFT & FFT Online Test Survival Guide

Theory, reusable coding snippets, debugging traps, and original practice problems

Built around the January 2026 offline, the provided `transforms.py`, `bigmul.py`, `image_conv.py`, and the uploaded online-test patterns.

What tomorrow's test is most likely to reward

The uploaded online questions do not ask you to re-write the entire offline. They give a new signal-processing story and expect you to **reuse the same few operations**: choose a valid transform size, pad correctly, transform, combine spectra, inverse-transform, then perform one final domain-specific step such as cropping, reversing a shift, carrying digits, or restoring coefficient order.

Contents

1	Source-Derived Pattern: What Has Been Asked	4
2	The Core Transform Engine You Already Have	4
2.1	DFT and IDFT	4
2.2	Naive DFT complexity	4
2.3	Radix-2 DIT FFT	5
2.4	IFFT through the same FFT machinery	5
2.5	Power-of-two helper	5
3	Convolution: The Main Theorem Behind Almost Everything	5
3.1	Convolution theorem	5
3.2	Linear vs circular convolution	6
3.3	One-dimensional reusable linear convolution	6
4	Two-Dimensional DFT and Image Convolution	7
4.1	2D DFT	7
4.2	Reusable row-column transform	7
4.3	2D linear-convolution size	7
4.4	Why the crop starts at half the kernel	8
4.5	Reusable linear image convolution	8
4.6	Circular image convolution and kernel rolling	8
4.7	Grayscale and RGB wrapper	9
5	Low-Pass, High-Pass, Hybrid Images, and Spectral Algebra	9
5.1	Low and high frequencies	9
5.2	Why the current hybrid question uses $\Delta - G$ rather than simply $1 - G$	9
5.3	Current-year hybrid image formula	9
5.4	Reusable hybrid-plane implementation pattern	10
5.5	One-inverse principle	10
6	Correlation and Shift Detection	10
6.1	Convolution vs correlation	10
6.2	Circular shift property	11
6.3	Detecting a row shift	11
6.4	Convert a wrapped peak to a signed shift	11
6.5	2D circular registration	11
6.6	Phase correlation: useful extra technique	11
7	Polynomial and Huge-Integer Patterns	12

7.1	Coefficient ordering is a major exam trap	12
7.2	Weighted polynomial pattern	12
7.3	Huge integers as polynomial coefficients	12
7.4	Carry propagation	13
7.5	Why rounding is safe only within a bound	13
8	NumPy / Python Snippets Worth Memorizing	13
9	Code-Specific Traps in the Provided Files	14
9.1	Trap 1: <code>isinstance(engine, FFTTransformer)</code> and inheritance	14
9.2	Trap 2: DFT does not require power-of-two padding	14
9.3	Trap 3: trim to the useful convolution length, not transform length	14
9.4	Trap 4: kernel alignment convention	14
9.5	Trap 5: correlation sign	14
9.6	Trap 6: integer vs floating output	14
9.7	Trap 7: RGB processing	15
9.8	Trap 8: forgetting the one-inverse requirement	15
10	A 90-Second Strategy for Any New Online Question	15
11	Original Practice Problem 1: Spectral Unsharp Mask	15
12	Original Practice Problem 2: Difference-of-Gaussians Edge Band	16
13	Original Practice Problem 3: Column-Wise Vertical Jitter Repair	17
14	Original Practice Problem 4: Full 2D Circular Image Registration	18
15	Original Practice Problem 5: Compute $xP'(x)Q(x)$ by FFT	18
16	Original Practice Problem 6: Base-100 Digit Multiplication	19
17	Original Practice Problem 7: Circular Autocorrelation for Repetition Lag	20
18	Original Practice Problem 8: Adjustable Hybrid Strength	20
19	Original Practice Problem 9: Blend Two Blurs with One IFFT	21
20	Original Practice Problem 10: Apply Two Linear Filters with One IFFT	22
21	Original Practice Problem 11: Locate a Small Template by 2D FFT Correlation	23
22	Original Practice Problem 12: Correct RGB Channel Misalignment	24
23	Original Practice Problem 13: Remove Periodic Stripe Noise with a Notch Mask	25
24	Original Practice Problem 14: Shift an Image Using Only a Phase Ramp	25
25	Original Practice Problem 15: Create a Circular Echo by Spectral Multiplication	26
26	Original Practice Problem 16: Stabilized Circular Deblurring	27
27	Original Practice Problem 17: Sum Two Polynomial Products with One IFFT	27
28	Original Practice Problem 18: Horizontal Edge Map from an Analytic DFT Filter	28
29	Mini Worked Examples You Should Be Able to Do by Hand	29
29.1	Linear vs circular convolution	29

29.2 Descending polynomial order	29
29.3 Correlation shift sign	30
29.4 2D crop indices	30
30 Debugging by Symptom	30
31 Fast Self-Tests Before the Exam	30
31.1 Transform agreement	30
31.2 Convolution sanity check without forbidden libraries	31
31.3 Correlation sanity check	31
31.4 2D shape sanity check	31
32 Likely New Question Families and the One-Line Recognition Rule	31
33 Last-Minute Cheat Sheet	32
Basis of This Guide	33

1 Source-Derived Pattern: What Has Been Asked

The offline establishes one shared transform core and two applications: huge-integer multiplication and image convolution. The online samples then modify these ideas instead of asking for the offline verbatim.

Question family	Core mathematical idea	Coding modification usually required
Huge integer digits	Polynomial multiplication = linear convolution	Pad, FFT/IFFT, round, carry propagation
Weighted polynomial	Pre-weight one coefficient sequence, then convolve	Reverse descending coefficients, multiply by weights, FFT/IFFT, trim, reverse result
Rolling-shutter correction	Circular shift is detected by FFT cross-correlation	Conjugate one spectrum, IFFT, <code>argmax</code> , undo shift with <code>np.roll</code>
Hybrid image (Jan 2026)	Low-pass from one image + high-pass from another	Build spectral expression $Y = LG + H(\Delta - G)$, exactly one inverse, crop, handle gray/RGB

Inference for preparation

The safest preparation is not memorizing one finished program. Memorize the **spectral building blocks** and recognize which one the problem statement describes. A new problem can often be solved by changing only the expression between forward transforms and the inverse transform.

2 The Core Transform Engine You Already Have

2.1 DFT and IDFT

For a length- N sequence $x[n]$,

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1.$$

The inverse is

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{+j2\pi kn/N}.$$

In your provided code, the sign convention is exactly the standard convention above: forward uses a negative exponent; inverse uses a positive exponent and divides by N .

Why this matters in coding questions

The sign convention determines correlation formulas and shift direction. With your convention, if `shifted = np.roll(reference, s)`, then

$$r = \text{IDFT}\{\overline{X_{ref}} X_{shifted}\}$$

has its peak at index s for circular shifts. To repair the shifted signal, roll it by $-s$.

2.2 Naive DFT complexity

A literal DFT has two loops: one over output bin k , one over input sample n . Therefore

$$T(N) = O(N^2).$$

The direct DFT is useful as a correctness reference, but it does not make convolution asymptotically faster: two $O(N^2)$ transforms still dominate.

2.3 Radix-2 DIT FFT

Your `FFTTransformer` is iterative radix-2 decimation-in-time (DIT). The structure is:

1. Input length must be a power of two.
2. Bit-reverse the input order.
3. Run $\log_2 N$ stages.
4. Stage s has group size $M = 2^s$ and half-size $M/2$.
5. Compute the stage twiddles once:

$$W_M^k = e^{-j2\pi k/M}, \quad 0 \leq k < M/2.$$

6. Each butterfly performs

$$\begin{aligned} g &= x[a], & h &= W_M^k x[b], \\ x[a] &\leftarrow g + h, & x[b] &\leftarrow g - h. \end{aligned}$$

Complexity:

$$O(N \log N).$$

2.4 IFFT through the same FFT machinery

Your code uses the conjugation identity

$$\text{IDFT}(X) = \frac{1}{N} \overline{\text{DFT}(\bar{X})}.$$

Reusable form:

```
1 def inverse(self, spectrum):
2     N = len(spectrum)
3     temp = self.transform(np.conjugate(spectrum))
4     return np.conjugate(temp) / N
```

This is worth remembering because an exam may ask you to avoid writing a second butterfly implementation.

2.5 Power-of-two helper

For your radix-2 FFT, the required length is often

$$N = 2^{\lceil \log_2 L \rceil}.$$

Use the provided helper rather than re-deriving it inside every online solution:

```
1 N = next_power_of_two(L)
```

3 Convolution: The Main Theorem Behind Almost Everything

3.1 Convolution theorem

For sequences a and b ,

$$a * b = \text{IDFT}\{\text{DFT}(a) \text{DFT}(b)\}.$$

The multiplication in frequency is **pointwise**:

```
1 A = engine.transform(a_pad)
2 B = engine.transform(b_pad)
3 C = A * B
4 c = engine.inverse(C)
```

3.2 Linear vs circular convolution

A length- N DFT naturally produces **circular** convolution:

$$(a \otimes_N b)[m] = \sum_{i=0}^{N-1} a[i] b[(m - i) \bmod N].$$

But ordinary polynomial multiplication and zero-outside-image convolution require **linear** convolution.

If lengths are n and q , the linear result length is

$$L = n + q - 1.$$

Therefore choose

$$N \geq L.$$

For your radix-2 FFT:

$$N = \text{next_power_of_two}(L).$$

For the naive DFT or the arbitrary-length FFT, $N = L$ is sufficient.

```

1 def choose_length(na, nb, engine):
2     L = na + nb - 1
3     if engine.name == "fft":
4         return L, next_power_of_two(L)
5     return L, L

```

The most common wrong answer

Using $N = \max(n, q)$ is not enough for linear convolution. The high-order part wraps around and contaminates the beginning. The output can still look plausible, which makes this bug dangerous.

3.3 One-dimensional reusable linear convolution

```

1 def linear_conv_1d(a, b, engine, integer_output=False):
2     a = np.asarray(a)
3     b = np.asarray(b)
4
5     L = len(a) + len(b) - 1
6     N = next_power_of_two(L) if engine.name == "fft" else L
7
8     ap = np.pad(a, (0, N - len(a)))
9     bp = np.pad(b, (0, N - len(b)))
10
11     y = engine.inverse(
12         engine.transform(ap) * engine.transform(bp)
13     ).real
14
15     y = y[:L]
16
17     if integer_output:
18         y = np rint(y).astype(np.int64)
19
20     return y

```

Round only when mathematics says the answer is integer

For big-integer multiplication, the true convolution coefficients are integers, so `np rint(...).astype(np.int64)` is correct. For ordinary polynomial coefficients or image pixels, do **not** force integer rounding unless the question explicitly requires it.

4 Two-Dimensional DFT and Image Convolution

4.1 2D DFT

For an image/plane $f[r, c]$ of shape $P \times Q$,

$$F[u, v] = \sum_{r=0}^{P-1} \sum_{c=0}^{Q-1} f[r, c] e^{-j2\pi(ur/P + vc/Q)}.$$

The exponential separates:

$$e^{-j2\pi(ur/P + vc/Q)} = e^{-j2\pi ur/P} e^{-j2\pi vc/Q}.$$

Therefore transform every row, then every column. This is exactly how your `image_conv.py` is structured.

4.2 Reusable row-column transform

```

1 def transform_2d(plane, engine):
2     R, C = plane.shape
3     temp = np.zeros((R, C), dtype=np.complex128)
4     out = np.zeros((R, C), dtype=np.complex128)
5
6     for r in range(R):
7         temp[r, :] = engine.transform(plane[r, :])
8
9     for c in range(C):
10        out[:, c] = engine.transform(temp[:, c])
11
12    return out
13
14
15 def inverse_2d(spec, engine):
16     R, C = spec.shape
17     temp = np.zeros((R, C), dtype=np.complex128)
18     out = np.zeros((R, C), dtype=np.complex128)
19
20     for r in range(R):
21         temp[r, :] = engine.inverse(spec[r, :])
22
23     for c in range(C):
24         out[:, c] = engine.inverse(temp[:, c])
25
26    return out

```

For a square $N \times N$ image:

$$\text{naive row-column DFT} = O(N^3), \quad \text{row-column FFT} = O(N^2 \log N).$$

4.3 2D linear-convolution size

For image shape (H, W) and kernel shape (k_h, k_w) ,

$$L_H = H + k_h - 1, \quad L_W = W + k_w - 1.$$

For radix-2 FFT:

$$T_H = \text{next_power_of_two}(L_H), \quad T_W = \text{next_power_of_two}(L_W).$$

```

1 def choose_shape(H, W, kh, kw, engine):
2     LH = H + kh - 1
3     LW = W + kw - 1
4
5     if engine.name == "fft":
6         return next_power_of_two(LH), next_power_of_two(LW)
7     return LH, LW

```

4.4 Why the crop starts at half the kernel

In the offline implementation, the kernel is placed at the top-left of the padded array before the transform. The full convolution is therefore offset relative to the desired same-size image. After IFFT, return

$$\begin{aligned} \text{rows } \lfloor k_h/2 \rfloor : \lfloor k_h/2 \rfloor + H, \\ \text{cols } \lfloor k_w/2 \rfloor : \lfloor k_w/2 \rfloor + W. \end{aligned}$$

```

1 r0 = kh // 2
2 c0 = kw // 2
3 same = full[r0:r0 + H, c0:c0 + W]

```

4.5 Reusable linear image convolution

```

1 def convolve_plane_linear(plane, kernel, engine):
2     H, W = plane.shape
3     kh, kw = kernel.shape
4     TH, TW = choose_shape(H, W, kh, kw, engine)
5
6     p = np.zeros((TH, TW), dtype=np.complex128)
7     k = np.zeros((TH, TW), dtype=np.complex128)
8     p[:H, :W] = plane
9     k[:kh, :kw] = kernel
10
11     P = transform_2d(p, engine)
12     K = transform_2d(k, engine)
13     full = inverse_2d(P * K, engine).real
14
15     r0, c0 = kh // 2, kw // 2
16     return full[r0:r0 + H, c0:c0 + W]

```

4.6 Circular image convolution and kernel rolling

For deliberate circular convolution, transform at exactly the image period (H, W) . To make the kernel's center correspond to zero displacement, roll the padded kernel:

```

1 k = np.zeros((H, W), dtype=np.complex128)
2 k[:kh, :kw] = kernel
3 k = np.roll(k, shift=(-kh // 2, -kw // 2), axis=(0, 1))

```

Then multiply spectra and inverse-transform without the linear crop.

Do not silently enlarge a circular-convolution period

If the required circular period is exactly (H, W) and the radix-2 FFT cannot transform those dimensions, padding to a larger size changes the circular-convolution period. Use a compatible engine or reject the unsupported shape. In the offline, the supplied circular-path images are already power-of-two sized.

4.7 Grayscale and RGB wrapper

This pattern appears repeatedly in online questions:

```

1 def apply_per_plane(image, plane_function):
2     if image.ndim == 2:
3         return plane_function(image)
4
5     out = np.zeros_like(image, dtype=np.float64)
6     for ch in range(image.shape[2]):
7         out[:, :, ch] = plane_function(image[:, :, ch])
8     return out

```

5 Low-Pass, High-Pass, Hybrid Images, and Spectral Algebra

5.1 Low and high frequencies

Low spatial frequencies describe slow variation: broad lighting, sky, smooth walls. High frequencies describe rapid variation: edges, texture, fine detail, and often noise.

A blur kernel is a low-pass filter. If G is its frequency response, multiplying an image spectrum by G suppresses high frequencies.

5.2 Why the current hybrid question uses $\Delta - G$ rather than simply $1 - G$

The current online problem defines Δ as the spectrum of a spatial impulse **aligned with the kernel center**. This is important because the implementation uses the same full-convolution alignment and crop as the offline image convolution.

Let the impulse array contain

$$\delta[k_h/2, k_w/2] = 1.$$

Then convolving with this impulse shifts the padded image exactly enough that the usual crop reconstructs the original plane. Thus, in this coordinate convention, the identity transfer function is Δ , and the complementary high-pass response is

$$\Delta - G.$$

If instead a kernel had already been rolled so that its center were at spatial index $(0,0)$ for circular convolution, the identity spectrum would simply be all ones. These are two different alignment conventions; do not mix them.

5.3 Current-year hybrid image formula

The uploaded January 2026 online question specifies

$$Y = LG + H(\Delta - G),$$

where L is the padded spectrum of the low-frequency source, H is the padded spectrum of the high-frequency source, and G is the Gaussian-kernel spectrum.

Interpretation:

LG = smooth component from low-source image,

$H(\Delta - G)$ = detail component from high-source image.

Only one inverse 2D transform is needed after adding the two spectral components.

5.4 Reusable hybrid-plane implementation pattern

```

1 def hybrid_plane(low, high, kernel, engine):
2     H, W = low.shape
3     kh, kw = kernel.shape
4     TH, TW = choose_shape(H, W, kh, kw, engine)
5
6     lp = np.zeros((TH, TW), dtype=np.complex128)
7     hp = np.zeros((TH, TW), dtype=np.complex128)
8     kp = np.zeros((TH, TW), dtype=np.complex128)
9     delta = np.zeros((TH, TW), dtype=np.complex128)
10
11     lp[:H, :W] = low
12     hp[:H, :W] = high
13     kp[:kh, :kw] = kernel
14     delta[kh // 2, kw // 2] = 1.0
15
16     L = transform_2d(lp, engine)
17     Hspec = transform_2d(hp, engine)
18     G = transform_2d(kp, engine)
19     Delta = transform_2d(delta, engine)
20
21     Y = L * G + Hspec * (Delta - G)
22     full = inverse_2d(Y, engine).real
23
24     r0, c0 = kh // 2, kw // 2
25     return full[r0:r0 + H, c0:c0 + W]

```

5.5 One-inverse principle

If several operations are linear and already expressed in the frequency domain, combine them **before** inverse-transforming.

Examples:

$$\begin{aligned}
 \text{blur: } Y &= XG, \\
 \text{high-pass: } Y &= X(\Delta - G), \\
 \text{hybrid: } Y &= LG + H(\Delta - G), \\
 \text{unsharp mask: } Y &= X[\Delta + \alpha(\Delta - G)].
 \end{aligned}$$

Then perform one IFFT.

6 Correlation and Shift Detection

6.1 Convolution vs correlation

Convolution in frequency:

$$y = x * h \iff Y = XH.$$

Cross-correlation introduces a conjugate. For the convention useful with your code,

$$r_{xy} = \text{IDFT}\{\overline{X}Y\}.$$

Memory rule

Convolution: multiply. Correlation: conjugate one side, then multiply.

6.2 Circular shift property

If

$$y[n] = x[(n - s) \bmod N],$$

which is what `np.roll(x, s)` produces, then

$$Y[k] = X[k]e^{-j2\pi ks/N}.$$

The magnitude spectrum is unchanged; the shift appears in phase.

6.3 Detecting a row shift

```
1 def detect_circular_shift(reference, shifted, engine):
2     X = engine.transform(reference)
3     Y = engine.transform(shifted)
4     corr = engine.inverse(np.conjugate(X) * Y).real
5     return int(np.argmax(corr))
```

If the result is s , repair with

```
1 fixed = np.roll(shifted, -s)
```

6.4 Convert a wrapped peak to a signed shift

A circular peak index may represent a negative displacement. Convert

```
1 def signed_shift(k, N):
2     return k if k <= N // 2 else k - N
```

For example, with $N = 8$, peak 7 means shift -1 .

6.5 2D circular registration

For two same-size planes where `moved = np.roll(ref, (dr, dc), axis=(0,1))`:

```
1 def detect_2d_shift(ref, moved, engine):
2     F = transform_2d(ref, engine)
3     M = transform_2d(moved, engine)
4     corr = inverse_2d(np.conjugate(F) * M, engine).real
5
6     pr, pc = np.unravel_index(np.argmax(corr), corr.shape)
7     H, W = ref.shape
8
9     dr = pr if pr <= H // 2 else pr - H
10    dc = pc if pc <= W // 2 else pc - W
11    return dr, dc
```

Repair:

```
1 fixed = np.roll(moved, shift=(-dr, -dc), axis=(0, 1))
```

6.6 Phase correlation: useful extra technique

If brightness or overall spectral amplitude differs, normalized cross-power can make shift detection more robust:

```
1 R = np.conjugate(F) * M
2 R = R / np.maximum(np.abs(R), 1e-12)
3 corr = inverse_2d(R, engine).real
```

This keeps phase information while reducing dependence on magnitude. Use it only if the problem allows or asks for phase correlation; ordinary cross-correlation is simpler when exact copies are shifted.

7 Polynomial and Huge-Integer Patterns

7.1 Coefficient ordering is a major exam trap

The transform convolution formula expects index i to mean coefficient of x^i :

$$[p_0, p_1, \dots, p_m].$$

But a question may provide the conventional written order

$$[p_m, p_{m-1}, \dots, p_0].$$

Then reverse before convolution:

```
1 p = np.asarray(P_desc)[::-1]
2 q = np.asarray(Q_desc)[::-1]
```

After convolution, reverse again if the requested answer is descending-power order.

7.2 Weighted polynomial pattern

If

$$R[k] = \sum_i w_i p_i q_{k-i},$$

define

$$a_i = w_i p_i.$$

Then

$$R = a * q.$$

```
1 p = np.asarray(P_desc, dtype=float)[::-1]
2 q = np.asarray(Q_desc, dtype=float)[::-1]
3 w = np.asarray(W_desc, dtype=float)[::-1]
4
5 a = p * w
6 r_asc = linear_conv_1d(a, q, engine, integer_output=False)
7 r_desc = r_asc[::-1]
```

A bug that looks harmless

It is easy to compute $\mathbf{a} = \mathbf{p} * \mathbf{w}$ correctly and then accidentally call the convolution with $\mathbf{p}$ instead of $\mathbf{a}$. Always trace the variable that should enter the transform.

7.3 Huge integers as polynomial coefficients

Write a large number in base B :

$$A = \sum_i a_i B^i, \quad C = \sum_j c_j B^j.$$

Then

$$AC = \sum_m \left(\sum_{i+j=m} a_i c_j \right) B^m.$$

The inner coefficient sum is convolution.

The offline packs four decimal digits per limb:

$$B = 10^4.$$

Example:

$$123456789 \rightarrow [6789, 2345, 1].$$

This is least-significant limb first.

7.4 Carry propagation

After IFFT, the raw convolution coefficients are not normalized base- B digits.

```

1 def propagate_carry(coeff, base):
2     out = []
3     carry = 0
4
5     for value in coeff:
6         total = int(value) + carry
7         out.append(total % base)
8         carry = total // base
9
10    while carry > 0:
11        out.append(carry % base)
12        carry //= base
13
14    while len(out) > 1 and out[-1] == 0:
15        out.pop()
16
17    return out

```

7.5 Why rounding is safe only within a bound

Double precision has a 53-bit significand, so integers are represented exactly only up to about

$$2^{53} \approx 9 \times 10^{15}.$$

A convolution coefficient is bounded approximately by

$$p_{\max} \leq n(B-1)^2.$$

A larger base shortens the transform but makes coefficients larger. This is why the offline chooses a conservative base.

8 NumPy / Python Snippets Worth Memorizing

Goal	Snippet / meaning
Reverse coefficient order	<code>a = a[::-1]</code>
Pad 1D to length N	<code>np.pad(a, (0, N-len(a)))</code>
Create padded 2D array	<code>p = np.zeros((TH,TW), dtype=np.complex128)</code>
Place image in padded array	<code>p[:H, :W] = image</code>
Conjugate spectrum	<code>np.conjugate(X)</code>
Remove tiny imaginary residue	<code>y = y.real</code>
Round mathematically integral output	<code>np.rint(y).astype(np.int64)</code>
Find 1D correlation peak	<code>shift = int(np.argmax(corr))</code>
Find 2D peak	<code>r,c = np.unravel_index(np.argmax(corr), corr.shape)</code>
Circularly shift	<code>np.roll(x, shift)</code>
Shift 2D	<code>np.roll(img, (dr,dc), axis=(0,1))</code>
Move kernel center to origin	<code>np.roll(k, (-kh//2,-kw//2), axis=(0,1))</code>
Maximum verification error	<code>np.max(np.abs(a-b))</code>
Tolerance comparison	<code>np.allclose(a, b, atol=1e-9)</code>
RGB channel count	<code>image.shape[2]</code>
Transpose columns iterator	<code>for c, col in enumerate(matrix.T):</code>

9 Code-Specific Traps in the Provided Files

9.1 Trap 1: `isinstance(engine, FFTTransformer)` and inheritance

Your optional `ArbitraryLengthFFT` inherits from `FFTTransformer`. Therefore

```
1 isinstance(ArbitraryLengthFFT(), FFTTransformer)
```

is true. If you use that test to decide “must pad to a power of two”, you also accidentally power-of-two pad the arbitrary-length engine.

Prefer:

```
1 if engine.name == "fft":
2     # radix-2 restriction only
```

when the distinction is about transform-length capability.

9.2 Trap 2: DFT does not require power-of-two padding

For linear convolution, the naive DFT can use the exact required length $L = n + q - 1$. Padding it further to a power of two is not incorrect, but it performs unnecessary work.

9.3 Trap 3: trim to the useful convolution length, not transform length

If $L = 6$ and radix-2 FFT uses $N = 8$, the valid linear-convolution coefficients are the first six values. Do not return all eight just because the IFFT produced eight samples.

```
1 y = y[:L]           # correct
2 # y = y[:N]         # contains padding tail when N > L
```

9.4 Trap 4: kernel alignment convention

Linear full convolution in the offline: kernel at top-left, then crop by half kernel.

Circular convolution: kernel center rolled to origin, then no linear crop.

Hybrid current-year question: centered impulse Δ is used so that the standard linear crop represents identity correctly.

9.5 Trap 5: correlation sign

With

```
1 shifted = np.roll(reference, s)
```

and

```
1 corr = inverse(conj(FFT(reference)) * FFT(shifted))
```

the detected peak is s ; repair using $-s$. If you reverse which spectrum is conjugated, the sign reverses.

9.6 Trap 6: integer vs floating output

- Big-integer raw coefficients: round to nearest integer.
- Weighted polynomial with arbitrary real coefficients: usually keep float.
- Image filtering: keep float; image-saving code can clip/scale as needed.

9.7 Trap 7: RGB processing

Do not transform a shape $(H, W, 3)$ with a helper designed for 2D. Process each channel independently unless the question explicitly defines cross-channel processing.

9.8 Trap 8: forgetting the one-inverse requirement

If a question gives several spectral terms and explicitly says “one inverse”, build the full spectral expression first. Performing separate inverses and adding later is mathematically possible for linear operations, but violates the requested implementation and wastes transforms.

10 A 90-Second Strategy for Any New Online Question

1. **Read the TODO signature first.** What is already prepared for you: spatial arrays, spectra, kernel, dimensions, engine?
2. **Identify dimension.** 1D sequence, 2D plane, or RGB wrapper?
3. **Identify operation.**
 - convolution $\Rightarrow XH$,
 - correlation $\Rightarrow \overline{X}Y$,
 - blur $\Rightarrow XG$,
 - high-pass $\Rightarrow X(\Delta - G)$,
 - hybrid $\Rightarrow LG + H(\Delta - G)$.
4. **Check ordering.** LSD-first? MSD-first? ascending or descending polynomial powers?
5. **Compute required length/shape.** For linear convolution, add sizes and subtract one.
6. **Apply radix-2 rule only to `engine.name == "fft"`.**
7. **Pad consistently.** All multiplied spectra must have identical shapes.
8. **Transform.**
9. **Do the spectral algebra.**
10. **Inverse once when possible.**
11. **Post-process.** `.real`, crop, trim, carry, reverse order, or undo shift.
12. **Handle grayscale/RGB.**
13. **Test a tiny case before running the full input.**

11 Original Practice Problem 1: Spectral Unsharp Mask

Problem

You are given an image plane X and a Gaussian kernel g . Build a sharpened result using

$$y = x + \alpha(x - x * g), \quad \alpha > 0.$$

You must use the provided 2D transform helpers and perform **exactly one inverse 2D transform**. Return an output plane with the same shape as the input.

Reasoning

With the offline linear-convolution alignment, identity is represented by the centered impulse spectrum Δ :

$$Y = X\Delta + \alpha X(\Delta - G) = X[\Delta + \alpha(\Delta - G)].$$

Solution

```

1 def sharpen_plane(plane, kernel, alpha, engine):
2     H, W = plane.shape
3     kh, kw = kernel.shape
4     TH, TW = choose_shape(H, W, kh, kw, engine)
5
6     p = np.zeros((TH, TW), dtype=np.complex128)
7     k = np.zeros((TH, TW), dtype=np.complex128)
8     d = np.zeros((TH, TW), dtype=np.complex128)
9
10    p[:H, :W] = plane
11    k[:kh, :kw] = kernel
12    d[kh // 2, kw // 2] = 1.0
13
14    X = transform_2d(p, engine)
15    G = transform_2d(k, engine)
16    Delta = transform_2d(d, engine)
17
18    Y = X * (Delta + alpha * (Delta - G))
19    full = inverse_2d(Y, engine).real
20
21    r0, c0 = kh // 2, kw // 2
22    return full[r0:r0 + H, c0:c0 + W]
```

What this tests

Transform-shape choice, identity alignment, spectral algebra, one-IFFT optimization, and correct crop.

12 Original Practice Problem 2: Difference-of-Gaussians Edge Band

Problem

Two Gaussian kernels g_1 and g_2 have the **same shape**, but g_1 is narrower and g_2 is wider. Produce

$$y = x * g_1 - x * g_2$$

with one inverse transform. This isolates a band of spatial frequencies useful for edge/detail enhancement.

Solution

```

1 def dog_plane(plane, g1, g2, engine):
2     H, W = plane.shape
3     kh, kw = g1.shape
4     assert g1.shape == g2.shape
5
6     TH, TW = choose_shape(H, W, kh, kw, engine)
7
8     p = np.zeros((TH, TW), dtype=np.complex128)
9     k1 = np.zeros((TH, TW), dtype=np.complex128)
10    k2 = np.zeros((TH, TW), dtype=np.complex128)
11
12    p[:H, :W] = plane
13    k1[:kh, :kw] = g1
```

```

14     k2[:kh, :kw] = g2
15
16     X = transform_2d(p, engine)
17     G1 = transform_2d(k1, engine)
18     G2 = transform_2d(k2, engine)
19
20     Y = X * (G1 - G2)
21     full = inverse_2d(Y, engine).real
22
23     r0, c0 = kh // 2, kw // 2
24     return full[r0:r0 + H, c0:c0 + W]

```

Why the question says same kernel shape

If two kernels have different support sizes and you place both at the top-left, their centers are at different spatial indices. Subtracting their spectra directly then mixes responses with different phase shifts. A common support/alignment must be established first.

13 Original Practice Problem 3: Column-Wise Vertical Jitter Repair

Problem

Instead of rows being shifted horizontally, suppose each **column** of an image was circularly shifted vertically by a different amount. A clean reference image is available. Detect each column's vertical shift using FFT-based correlation and reconstruct the image.

Solution

```

1 def repair_vertical_jitter(reference, damaged, engine):
2     H, W = reference.shape[:2]
3     fixed = np.empty_like(damaged)
4
5     for c in range(W):
6         if reference.ndim == 2:
7             ref_signal = reference[:, c]
8             bad_signal = damaged[:, c]
9         else:
10             # one scalar signal for shift detection
11             ref_signal = reference[:, c, :].mean(axis=1)
12             bad_signal = damaged[:, c, :].mean(axis=1)
13
14         R = engine.transform(ref_signal)
15         B = engine.transform(bad_signal)
16         corr = engine.inverse(np.conjugate(R) * B).real
17         shift = int(np.argmax(corr))
18
19         if reference.ndim == 2:
20             fixed[:, c] = np.roll(damaged[:, c], -shift)
21         else:
22             fixed[:, c, :] = np.roll(
23                 damaged[:, c, :], -shift, axis=0
24             )
25
26     return fixed

```

Key recognition

Same correlation idea as row shift, but the extracted 1D signals and roll axis change.

14 Original Practice Problem 4: Full 2D Circular Image Registration

Problem

A second image is a circular translation of a reference by an unknown $(\Delta r, \Delta c)$. Determine the shift with one 2D correlation and realign the image.

Solution

```

1 def register_circular(ref, moved, engine):
2     if ref.ndim == 3:
3         ref_plane = ref.mean(axis=2)
4         moved_plane = moved.mean(axis=2)
5     else:
6         ref_plane = ref
7         moved_plane = moved
8
9     F = transform_2d(ref_plane, engine)
10    M = transform_2d(moved_plane, engine)
11
12    corr = inverse_2d(np.conjugate(F) * M, engine).real
13    pr, pc = np.unravel_index(np.argmax(corr), corr.shape)
14
15    H, W = ref_plane.shape
16    dr = pr if pr <= H // 2 else pr - H
17    dc = pc if pc <= W // 2 else pc - W
18
19    return np.roll(moved, shift=(-dr, -dc), axis=(0, 1)), (dr, dc)

```

Tiny check

If `moved = np.roll(ref, (2,-3), axis=(0,1))`, the detector should return approximately $(2, -3)$ and the repair should roll by $(-2, +3)$.

15 Original Practice Problem 5: Compute $xP'(x)Q(x)$ by FFT

Problem

Polynomials are supplied in descending-power arrays. Compute

$$S(x) = xP'(x)Q(x)$$

using FFT/IFFT convolution. Return descending-power coefficients.

Reasoning

If

$$P(x) = \sum_{i=0}^m p_i x^i,$$

then

$$xP'(x) = \sum_{i=0}^m i p_i x^i.$$

So this is a weighted polynomial product with weights $w_i = i$.

Solution

```

1 def x_derivative_product(P_desc, Q_desc, engine):
2     p = np.asarray(P_desc, dtype=float)[::-1]
3     q = np.asarray(Q_desc, dtype=float)[::-1]
4
5     weights = np.arange(len(p), dtype=float)
6     a = weights * p
7
8     s_asc = linear_conv_1d(a, q, engine, integer_output=False)
9     return s_asc[::-1]

```

Example

Let

$$P(x) = 2x^2 + 3x + 4, \quad Q(x) = x + 2.$$

Then

$$xP'(x) = 4x^2 + 3x,$$

and

$$S(x) = (4x^2 + 3x)(x + 2) = 4x^3 + 11x^2 + 6x.$$

Expected descending array:

$$[4, 11, 6, 0].$$

16 Original Practice Problem 6: Base-100 Digit Multiplication

Problem

Two non-negative integers are supplied as least-significant-first base-100 digits, e.g.

$$A = [34, 12] \Rightarrow 1234.$$

Multiply them using FFT convolution and return normalized base-100 digits.

Solution

```

1 def multiply_base100(a, b, engine):
2     raw = linear_conv_1d(a, b, engine, integer_output=True)
3
4     base = 100
5     out = []
6     carry = 0
7
8     for v in raw:
9         total = int(v) + carry
10        out.append(total % base)
11        carry = total // base
12
13    while carry:
14        out.append(carry % base)
15        carry //= base
16
17    while len(out) > 1 and out[-1] == 0:
18        out.pop()
19
20    return out

```

What changed from the offline

The transform logic is identical. Only the base and input/output representation changed.

17 Original Practice Problem 7: Circular Autocorrelation for Repetition Lag

Problem

A length- N periodic sensor signal contains a repeating pattern. Compute its circular autocorrelation with FFT and report the strongest nonzero lag.

Theory

Autocorrelation is

$$r_{xx} = \text{IDFT}\{\overline{X}X\} = \text{IDFT}\{|X|^2\}.$$

Solution

```

1 def strongest_nonzero_lag(x, engine):
2     X = engine.transform(x)
3     corr = engine.inverse(np.conjugate(X) * X).real
4
5     # Ignore lag 0, which is normally the largest energy peak.
6     lag = int(np.argmax(corr[1:]) + 1)
7
8     # Optional signed interpretation
9     if lag > len(x) // 2:
10         lag -= len(x)
11
12     return lag, corr

```

Exam connection

This is the same conjugate-spectrum machinery as image alignment, but both inputs are the same signal.

18 Original Practice Problem 8: Adjustable Hybrid Strength

Problem

Create a hybrid plane with a user-controlled detail strength β :

$$Y = LG + \beta H(\Delta - G).$$

Use one inverse transform, preserve image size, and support gray/RGB.

Solution idea

Take the current hybrid pattern and change only the spectral algebra:

```

1 Y = L * G + beta * Hspec * (Delta - G)

```

Everything else—shape selection, padding, impulse alignment, one IFFT, crop, and per-channel wrapper—remains the same.

RGB wrapper

```

1 def hybrid_image(low_img, high_img, kernel, beta, engine):
2     if low_img.shape != high_img.shape:
3         raise ValueError("input images must have same shape")
4
5     if low_img.ndim == 2:
6         return hybrid_plane_beta(
7             low_img, high_img, kernel, beta, engine
8         )
9
10    out = np.zeros_like(low_img, dtype=np.float64)
11    for ch in range(low_img.shape[2]):
12        out[:, :, ch] = hybrid_plane_beta(
13            low_img[:, :, ch],
14            high_img[:, :, ch],
15            kernel, beta, engine
16        )
17    return out

```

19 Original Practice Problem 9: Blend Two Blurs with One IFFT

Problem

An image plane I must be filtered by two kernels h_1 and h_2 , then blended:

$$Y = \alpha(I * h_1) + (1 - \alpha)(I * h_2), \quad 0 \leq \alpha \leq 1.$$

Implement the result through the provided DFT/FFT routines using **exactly one inverse 2D transform**. Return an output with the same size as the input image. Assume both kernels have the same odd dimensions.

Recognition

Use linearity before the inverse transform:

$$Y = \text{IDFT} \{F(\alpha H_1 + (1 - \alpha)H_2)\}.$$

Do not compute two blurred images and blend them afterward; that would waste an inverse transform.

Solution

Pad everything to a common shape large enough for *both* kernels.

```

1 def blend_two_blurs(plane, k1, k2, alpha, engine):
2     H, W = plane.shape
3     h1, w1 = k1.shape
4     h2, w2 = k2.shape
5     if k1.shape != k2.shape:
6         raise ValueError("kernels must have the same shape")
7
8     full_h = H + h1 - 1
9     full_w = W + w1 - 1
10
11    if engine.name == "fft":
12        P = next_power_of_two(full_h)
13        Q = next_power_of_two(full_w)
14    else:
15        P, Q = full_h, full_w
16

```

```

17 Ipad = np.zeros((P, Q), dtype=np.complex128)
18 K1 = np.zeros((P, Q), dtype=np.complex128)
19 K2 = np.zeros((P, Q), dtype=np.complex128)
20
21 Ipad[:, :H] = plane
22 K1[:, :h1] = k1
23 K2[:, :h2] = k2
24
25 F = transform_2d(Ipad, engine)
26 H1 = transform_2d(K1, engine)
27 H2 = transform_2d(K2, engine)
28
29 Yspec = F * (alpha * H1 + (1-alpha) * H2)
30 full = inverse_2d(Yspec, engine).real
31
32 sr = h1 // 2
33 sc = w1 // 2
34 return full[sr:sr+H, sc:sc+W]

```

Alignment detail

If an exam gives kernels with unequal shapes, their centres must be aligned consistently before the spectra are combined. The easiest safe version of this problem uses equal-shaped kernels; otherwise you must account for the relative spatial shift before using one shared crop.

20 Original Practice Problem 10: Apply Two Linear Filters with One IFFT

Problem

You are given an image I and two odd-sized kernels h_1 and h_2 . Compute the **full sequential linear filtering**

$$Y = (I * h_1) * h_2$$

without materializing the intermediate image and with only one inverse 2D transform. Then return the centred (H, W) crop.

Reasoning

Convolution is associative and transforms convert each convolution into multiplication:

$$\text{DFT}\{(I * h_1) * h_2\} = FH_1H_2.$$

The full output size is

$$(H + k_{1r} + k_{2r} - 2) \times (W + k_{1c} + k_{2c} - 2).$$

For odd kernels, the total centre offset is

$$s_r = \left\lfloor \frac{k_{1r}}{2} \right\rfloor + \left\lfloor \frac{k_{2r}}{2} \right\rfloor, \quad s_c = \left\lfloor \frac{k_{1c}}{2} \right\rfloor + \left\lfloor \frac{k_{2c}}{2} \right\rfloor.$$

Solution

```

1 def two_filters_one_inverse(plane, k1, k2, engine):
2     H, W = plane.shape
3     r1, c1 = k1.shape
4     r2, c2 = k2.shape
5
6     full_h = H + r1 + r2 - 2

```

```

7     full_w = W + c1 + c2 - 2
8
9     if engine.name == "fft":
10         P = next_power_of_two(full_h)
11         Q = next_power_of_two(full_w)
12     else:
13         P, Q = full_h, full_w
14
15     A = np.zeros((P, Q), dtype=np.complex128)
16     B = np.zeros((P, Q), dtype=np.complex128)
17     C = np.zeros((P, Q), dtype=np.complex128)
18     A[:H, :W] = plane
19     B[:r1, :c1] = k1
20     C[:r2, :c2] = k2
21
22     F = transform_2d(A, engine)
23     H1 = transform_2d(B, engine)
24     H2 = transform_2d(C, engine)
25
26     out = inverse_2d(F * H1 * H2, engine).real
27
28     sr = r1//2 + r2//2
29     sc = c1//2 + c2//2
30     return out[sr:sr+H, sc:sc+W]

```

Important distinction

This computes the full convolution chain and crops *once at the end*. If a problem explicitly crops back to (H, W) after the first filter and then filters that cropped image again, the boundary operation is different.

21 Original Practice Problem 11: Locate a Small Template by 2D FFT Correlation

Problem

A grayscale scene contains a smaller template at an unknown location. Locate the template using the provided 2D transform routines. You may not use `np.correlate`, `scipy.signal`, or OpenCV matching functions.

Assume the template appears without rotation or scaling and that the strongest correlation peak corresponds to its top-left displacement from the origin-aligned template.

Theory

Cross-correlation uses a conjugate instead of ordinary spectral multiplication:

$$R = \text{IDFT} \{ \overline{T} I \}.$$

For linear rather than circular matching, pad to at least

$$(H + k_h - 1) \times (W + k_w - 1).$$

Solution

```

1 def locate_template(scene, template, engine):
2     H, W = scene.shape
3     kh, kw = template.shape
4

```

```

5     req_h = H + kh - 1
6     req_w = W + kw - 1
7     if engine.name == "fft":
8         P = next_power_of_two(req_h)
9         Q = next_power_of_two(req_w)
10    else:
11        P, Q = req_h, req_w
12
13    A = np.zeros((P, Q), dtype=np.complex128)
14    B = np.zeros((P, Q), dtype=np.complex128)
15    A[:H, :W] = scene
16    B[:kh, :kw] = template
17
18    Ispec = transform_2d(A, engine)
19    Tspec = transform_2d(B, engine)
20
21    corr = inverse_2d(np.conjugate(Tspec) * Ispec, engine).real
22    r, c = np.unravel_index(np.argmax(corr), corr.shape)
23    return (int(r), int(c)), corr

```

Convolution is not correlation

$\text{Ispec} * \text{Tspec}$ performs convolution. Template matching needs one spectrum conjugated. Swapping which spectrum is conjugated reverses the measured displacement.

22 Original Practice Problem 12: Correct RGB Channel Misalignment

Problem

A colour image has three channels, but the red and blue planes were circularly shifted during capture. The green plane is correctly aligned. Estimate each channel's 2D shift using FFT correlation and repair the image.

Reasoning

Treat green as the reference. If

$$R = \text{roll}(G, (d_r, d_c)),$$

then

$$\text{IDFT}\{\overline{G_f} R_f\}$$

peaks at (d_r, d_c) modulo image size. Convert wrapped indices to signed shifts, then roll the misaligned channel by the negative shift.

Solution

```

1  def signed_shift(index, N):
2      return index - N if index > N//2 else index
3
4
5  def estimate_2d_shift(reference, shifted, engine):
6      A = transform_2d(reference, engine)
7      B = transform_2d(shifted, engine)
8      corr = inverse_2d(np.conjugate(A) * B, engine).real
9
10     r, c = np.unravel_index(np.argmax(corr), corr.shape)
11     return signed_shift(int(r), reference.shape[0]), \
12            signed_shift(int(c), reference.shape[1])
13
14

```

```

15 def align_rgb(image, engine):
16     out = image.copy()
17     G = image[:, :, 1]
18
19     for ch in (0, 2):
20         dr, dc = estimate_2d_shift(G, image[:, :, ch], engine)
21         out[:, :, ch] = np.roll(
22             image[:, :, ch], shift=(-dr, -dc), axis=(0, 1)
23         )
24     return out

```

Why this is likely exam material

It is the rolling-shutter question lifted from “one row at a time” to a full 2D channel. The only new step is finding a 2D peak with `np.unravel_index`.

23 Original Practice Problem 13: Remove Periodic Stripe Noise with a Notch Mask

Problem

An image contains a periodic stripe artifact. From inspection, its unwanted sinusoidal component is known to occupy DFT bin (u_0, v_0) . Remove that component in the frequency domain and reconstruct the image. The result should remain real-valued.

Theory

A real image has conjugate-symmetric DFT coefficients. Therefore a notch must suppress both

$$(u_0, v_0) \quad \text{and} \quad ((-u_0) \bmod H, (-v_0) \bmod W).$$

This is spectral multiplication by a mask $M[u, v]$.

Solution

```

1 def remove_periodic_bin(plane, u0, v0, engine, keep=0.0):
2     H, W = plane.shape
3     F = transform_2d(plane, engine)
4
5     mask = np.ones((H, W), dtype=np.float64)
6     mask[u0 % H, v0 % W] = keep
7     mask[(-u0) % H, (-v0) % W] = keep
8
9     cleaned = inverse_2d(F * mask, engine).real
10    return cleaned

```

Why two bins?

Zeroing only one member of a conjugate pair breaks the symmetry required for an exactly real inverse. You may then see a non-negligible imaginary component.

24 Original Practice Problem 14: Shift an Image Using Only a Phase Ramp

Problem

Circularly shift an image downward by d_r rows and rightward by d_c columns, but do not use `np.roll` for the actual shift. Use the DFT shift property and one inverse transform.

Theory

For your forward-transform sign convention,

$$y[r, c] = x[(r - d_r) \bmod H, (c - d_c) \bmod W]$$

corresponds to

$$Y[u, v] = X[u, v] \exp \left[-j2\pi \left(\frac{ud_r}{H} + \frac{vd_c}{W} \right) \right].$$

Solution

```

1 def spectral_roll(plane, dr, dc, engine):
2     H, W = plane.shape
3     F = transform_2d(plane, engine)
4
5     u = np.arange(H)[:, None]
6     v = np.arange(W)[None, :]
7     phase = np.exp(
8         -2j * np.pi * (u * dr / H + v * dc / W)
9     )
10
11     return inverse_2d(F * phase, engine).real

```

Self-test

For a small array, the answer should agree numerically with

```

1 np.roll(plane, shift=(dr, dc), axis=(0, 1))

```

Use `np.roll` only as a test if the online restrictions permit it; the implementation itself above does not need it.

25 Original Practice Problem 15: Create a Circular Echo by Spectral Multiplication

Problem

For a 1D signal $x[n]$ of length N , generate

$$y[n] = x[n] + a x[(n - d) \bmod N]$$

using one forward transform and one inverse transform. Do not explicitly construct the delayed signal.

Reasoning

The operation is circular convolution with

$$h[n] = \delta[n] + a\delta[n - d].$$

Its frequency response is

$$H[k] = 1 + ae^{-j2\pi kd/N}.$$

Therefore $Y[k] = X[k]H[k]$.

Solution

```

1 def circular_echo(x, delay, strength, engine):
2     x = np.asarray(x, dtype=np.complex128)
3     N = len(x)
4
5     X = engine.transform(x)
6     k = np.arange(N)
7     H = 1 + strength * np.exp(-2j * np.pi * k * delay / N)
8
9     return engine.inverse(X * H).real

```

Recognition rule

Whenever an output is a weighted sum of shifted copies of the same signal, think “impulse-response taps” and multiply by their DFT response.

26 Original Practice Problem 16: Stabilized Circular Deblurring

Problem

A grayscale image was blurred by **circular convolution** with a known kernel h . Recover an approximation of the original image. Directly dividing by the kernel spectrum is unstable when some frequency responses are near zero, so use

$$\hat{F} = Y \frac{\overline{H}}{|H|^2 + \lambda}, \quad \lambda > 0.$$

Solution

The kernel must be wrapped to the origin using the same convention as your circular-convolution code.

```

1 def circular_deblur(blurred, kernel, lam, engine):
2     H, W = blurred.shape
3     kh, kw = kernel.shape
4
5     Kpad = np.zeros((H, W), dtype=np.complex128)
6     Kpad[:kh, :kw] = kernel
7     Kpad = np.roll(
8         Kpad, shift=(-(kh//2), -(kw//2)), axis=(0, 1)
9     )
10
11     Y = transform_2d(blurred, engine)
12     K = transform_2d(Kpad, engine)
13
14     Fhat = Y * np.conjugate(K) / (np.abs(K)**2 + lam)
15     return inverse_2d(Fhat, engine).real

```

Why not simply use Y / K ?

If $|K[u, v]|$ is tiny, ordinary inverse filtering amplifies numerical error and noise enormously. The positive λ prevents division by values near zero.

27 Original Practice Problem 17: Sum Two Polynomial Products with One IFFT

Problem

Coefficient arrays A, B, C, D are supplied in ascending powers. Compute

$$R(x) = A(x)B(x) + C(x)D(x)$$

using FFT/DFT convolution, but perform only **one inverse transform**. Return the useful linear-convolution coefficients.

Reasoning

By linearity,

$$R = \text{IDFT}\{\widehat{A}\widehat{B} + \widehat{C}\widehat{D}\}.$$

All four arrays must use the same transform length, large enough for the longer product.

Solution

```

1 def sum_of_products(A, B, C, D, engine):
2     L1 = len(A) + len(B) - 1
3     L2 = len(C) + len(D) - 1
4     useful = max(L1, L2)
5
6     N = next_power_of_two(useful) if engine.name == "fft" else useful
7
8     def pad(x):
9         return np.pad(np.asarray(x, dtype=np.complex128),
10                        (0, N-len(x)))
11
12     FA = engine.transform(pad(A))
13     FB = engine.transform(pad(B))
14     FC = engine.transform(pad(C))
15     FD = engine.transform(pad(D))
16
17     result = engine.inverse(FA*FB + FC*FD).real
18     return result[:useful]
```

What this tests

The examiner can hide several convolutions inside one algebraic expression. Add or subtract their spectrum products first; inverse-transform only after the spectral expression is complete.

28 Original Practice Problem 18: Horizontal Edge Map from an Analytic DFT Filter

Problem

Create a circular horizontal finite-difference image

$$y[r, c] = x[r, c] - x[r, (c - 1) \bmod W]$$

without explicitly building or transforming a spatial kernel. Use the analytic frequency response and one inverse 2D transform.

Theory

The 1D horizontal kernel is

$$h = [1, -1],$$

so along column-frequency bin v ,

$$H[v] = 1 - e^{-j2\pi v/W}.$$

It does not depend on row-frequency bin u , so broadcasting can apply it to the whole 2D spectrum.

Solution

```
1 def horizontal_difference_fft(plane, engine):
2     H, W = plane.shape
3     F = transform_2d(plane, engine)
4
5     v = np.arange(W)[None, :]
6     response = 1 - np.exp(-2j * np.pi * v / W)
7
8     edges = inverse_2d(F * response, engine).real
9     return edges
```

Extension

A vertical finite difference uses

$$1 - e^{-j2\pi u/H}$$

with `u = np.arange(H)[: , None]`. A simple gradient magnitude can then be formed from the two reconstructed derivative planes.

What Problems 9–18 are training

These ten questions deliberately vary the *middle spectral expression*: weighted sums of filter responses, products of several filter responses, conjugate products for correlation, masks, phase ramps, analytic transfer functions, regularized inverse filtering, and sums of convolution products. The surrounding pipeline is still the same: choose shape → pad/alignment → forward transform → spectral algebra → inverse → crop/interpret.

29 Mini Worked Examples You Should Be Able to Do by Hand

29.1 Linear vs circular convolution

Let

$$a = [1, 2], \quad b = [3, 4].$$

Linear convolution:

$$[3, 10, 8].$$

If length $N = 2$ is used circularly, index 2 wraps onto index 0:

$$[3 + 8, 10] = [11, 10].$$

Therefore minimum linear length is $2 + 2 - 1 = 3$. Radix-2 FFT would use $N = 4$.

29.2 Descending polynomial order

Input

$$P_{desc} = [2, 3, 4]$$

means

$$2x^2 + 3x + 4.$$

Convolution index order must be

$$P_{asc} = [4, 3, 2].$$

Never multiply descending arrays directly unless the problem has explicitly defined its indexing that way.

29.3 Correlation shift sign

Suppose

$$y = \text{roll}(x, +2).$$

Using

$$r = \text{IDFT}\{\overline{XY}\},$$

the peak occurs at +2. Repair:

$$y_{\text{fixed}} = \text{roll}(y, -2).$$

29.4 2D crop indices

If image is 100×120 and kernel is 7×9 :

$$r_0 = 7//2 = 3, \quad c_0 = 9//2 = 4.$$

Return

```
1 full[3:103, 4:124]
```

which has shape 100×120 .

30 Debugging by Symptom

Symptom	Likely cause	First check
Beginning of 1D result is too large	Circular wrap-around	Is $N \geq n + q - 1$?
Image is diagonally shifted	Wrong/missing linear crop	Start at $kh//2, kw//2$
Circular blur looks shifted strangely	Kernel center not moved to origin	Check <code>np.roll</code> direction
FFT raises length error	Non-power-of-two dimension	Apply <code>next_power_of_two</code> for linear FFT
Arbitrary engine unexpectedly padded	Inheritance test caught subclass	Use <code>engine.name == "fft"</code>
Weighted polynomial looks like ordinary product	Weighted sequence unused	Convolve <code>p*w</code> , not <code>p</code>
Correct coefficients followed by zeros	Returned transform length, not useful length	Slice to $L = n + q - 1$
Real polynomial decimals disappear	Unnecessary integer cast	Remove <code>astype(np.int64)</code>
Shift correction doubles distortion	Correlation sign reversed	If peak is s , repair by $-s$ for chosen formula
RGB code crashes in 2D transform	Passed whole 3D image to plane function	Loop over channels
High-pass/hybrid is spatially misaligned	Used $1 - G$ with linear-crop convention	Build centered impulse spectrum Δ
Error around 10^{-15}	Floating-point roundoff	Normal; usually acceptable
Error above 10^{-9}	Structural bug	Recheck padding, crop, transform expression

31 Fast Self-Tests Before the Exam

31.1 Transform agreement

```

1 x = np.random.randn(64) + 1j * np.random.randn(64)
2 d = DFTAnalyzer()
3 f = FFTTransformer()
4
5 assert np.max(np.abs(d.transform(x) - f.transform(x))) < 1e-9
6 assert np.max(np.abs(f.inverse(f.transform(x)) - x)) < 1e-9

```

31.2 Convolution sanity check without forbidden libraries

For a tiny sequence, verify with loops:

```

1 def direct_conv(a, b):
2     out = np.zeros(len(a) + len(b) - 1)
3     for i in range(len(a)):
4         for j in range(len(b)):
5             out[i + j] += a[i] * b[j]
6     return out

```

31.3 Correlation sanity check

```

1 x = np.array([1., 4., 2., 7., 3., 0., 5., 6.])
2 y = np.roll(x, 3)
3
4 s = detect_circular_shift(x, y, FFTTransformer())
5 assert s == 3
6 assert np.allclose(np.roll(y, -s), x)

```

31.4 2D shape sanity check

After every plane operation:

```

1 assert result.shape == plane.shape

```

For RGB:

```

1 assert result.shape == image.shape

```

32 Likely New Question Families and the One-Line Recognition Rule

These are not copied questions; they are natural modifications of the same codebase.

Possible story	Recognition rule	Frequency-domain core
Sharpen an image	Original + scaled high-pass detail	$X[\Delta + \alpha(\Delta - G)]$
Extract edges/details	Remove Gaussian low-pass	$X(\Delta - G)$
Difference of two blur scales	Subtract two low-pass responses	$X(G_1 - G_2)$
Blend smooth A with detail B	Hybrid image	$AG + B(\Delta - G)$
Find circular translation	Correlation peak	$\text{IDFT}(\overline{AB})$
Find repeated lag	Autocorrelation	$\text{IDFT}(\overline{XX})$
Template/location search	Cross-correlation	conjugate template spectrum times signal spectrum
Weighted polynomial rule	Preprocess one sequence, then convolve	$\text{DFT}(a_{\text{weighted}}) \text{DFT}(b)$
Derivative-polynomial product	Weights are index-dependent	$a_i = ip_i$ (or related derivative weight)
Alternative digit base	Same convolution, different carry base	FFT product + base- B carry

33 Last-Minute Cheat Sheet

1D linear convolution

$$L = n + q - 1, \quad N = \begin{cases} \text{next_power_of_two}(L), & \text{engine.name} == \text{"fft"}, \\ L, & \text{otherwise.} \end{cases}$$

Pad $\rightarrow$ transform $\rightarrow$ multiply $\rightarrow$ inverse $\rightarrow$ real $\rightarrow$ first L samples.

2D linear convolution / filtering

$$L_H = H + k_h - 1, \quad L_W = W + k_w - 1.$$

Pad both axes. Transform rows then columns. Multiply spectra. IFFT. For same-size output crop from

$$(k_h/2, k_w/2).$$

Correlation

$$r = \text{IDFT}(\overline{XY}).$$

Peak $\Rightarrow$ displacement of Y relative to X under the convention used in this guide. Undo with the negative displacement.

Hybrid / high-pass with offline crop alignment

Centered impulse spectrum Δ is the identity transfer.

$$\text{low-pass} = G, \quad \text{high-pass} = \Delta - G,$$

$$\text{hybrid} = LG + H(\Delta - G).$$

Combine all spectral terms first; one inverse at the end.

Polynomial order

Question gives descending powers? Reverse to ascending before convolution. Reverse the result back only if the requested output is descending.

Huge integers

LSD/limb-first coefficients → FFT convolution → round integer coefficients → carry from low index upward.

Five checks before pressing Run

1. Did I distinguish convolution from correlation?
2. Is the transform size large enough, and only power-of-two padded when required?
3. Are all spectra the same shape?
4. Did I use the correct post-step: trim, crop, carry, reverse, or negative roll?
5. Did I accidentally round floats or pass RGB directly into a 2D-plane helper?

Basis of This Guide

This guide is grounded in the uploaded January 2026 CSE 220 offline specification, the provided implementations of `transforms.py`, `bigmul.py`, and `image_conv.py`, and the uploaded online-test examples covering huge-integer convolution, weighted polynomial convolution, row-wise FFT cross-correlation, and the January 2026 Fourier hybrid-image task. The eight practice problems in this document are newly constructed variants rather than copies of those test statements.